

NAME	Umair Noor
ROLL NO	2500603
CLASS	ADSCS-3A
SUBJECT	FULL STACK WEB-LAB
LAB	5

Task:
Interactive Student Registration Card

Build a single page (index.html, style.css, script.js) that lets a student register and see a live preview card. Reproduce the expected output shown on the slides.

1.
Create a form with fields: Full Name, Email, Program (select), Semester (number) and a Submit button.

2.

Select the form elements using getElementById and querySelector.

3.

Use an input event so the preview card updates live as the student types.

4.

On submit, call preventDefault(), validate all fields, and show an error message under any invalid field using textContent.

5.

If valid, create a new list item with createElement, append it to a "Registered Students" list, and clear the form.

6.

Toggle a CSS class (classList.toggle) to switch the card between light and dark mode with a button.

CODE :

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration Card</title>
  <link rel="stylesheet" href="style.css">
</head>
```

<body>

<div class="container">

<div class="form-section">

<h1>Student Registration</h1>

<form id="studentForm">

<label>Full Name</label>

<input type="text" id="fullName" placeholder="Enter your full name">

<p id="nameError" class="error"></p>

<label>Email</label>

<input type="email" id="email" placeholder="Enter your email">

<p id="emailError" class="error"></p>

<label>Program</label>

<select id="program">

<option value="">Select Program</option>

<option value="BS Computer Science">BS Computer Science</option>

<option value="BS Software Engineering">BS Software Engineering</option>

<option value="BS Information Technology">BS Information Technology</option>

<option value="BS Artificial Intelligence">BS Artificial Intelligence</option>

</select>

<p id="programError" class="error"></p>

<label>Semester</label>

<input type="number" id="semester" placeholder="Enter semester" min="1" max="8">

<p id="semesterError" class="error"></p>

<button type="submit">Register Student</button>

</form>

</div>

<div class="preview-section">

<button id="themeButton">Toggle Dark Mode</button>

<div id="studentCard" class="student-card">

<h2>Student Card</h2>

<p>Name: Your Name</p>

<p>Email: Your Email</p>

<p>Program: Your Program</p>

<p>Semester: Your Semester</p>

</div>

<h2>Registered Students</h2>

<ul id="studentList">

</div>

</div>

<script src="script.js"></script>

</body>

</html>

Style.css

```
* {  box-sizing: border-  
box;  
}
```

```
body {  margin: 0;  font-  
family: Arial, sans-serif;  
background-color: #f2f2f2;  
}
```

```
.container {  width:  
90%;  max-width:  
1000px;  margin:  
40px auto;  display:  
flex;  gap: 30px;  
}
```

```
.form-section,  
.preview-section {  background-color: white;  padding: 25px;  border-radius: 10px;  
  flex: 1;  
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.15);  
}
```

```
h1, h2 {  text-  
align: center;  
}
```

```
label {  display:  
block;  margin-  
top: 15px;  font-  
weight: bold;
```

```
}
```

```
input, select { width:  
100%; padding: 10px;  
margin-top: 5px;  
border: 1px solid #aaa;  
border-radius: 5px;  
}
```

```
button { width:  
100%; padding:  
11px; margin-top:  
15px; border:  
none; border-  
radius: 5px;  
cursor: pointer;  
background-color:  
#333; color:  
white;  
}
```

```
button:hover {  
opacity: 0.85;  
}
```

```
.error { color:  
red; font-size:  
14px; margin:  
5px 0; min-  
height: 16px;  
}
```

```
.student-card {  margin-top:
20px;  padding: 20px;
border-radius: 10px;
background-color: #eeeeee;
color: #222;  border: 2px
solid #555;
}
```

```
.student-card.dark {
background-color: #222;  color:
white;
}
```

```
#studentList {
padding: 0;  list-style:
none;
}
```

```
#studentList li {  padding:
10px;  margin-top: 8px;
background-color: #eeeeee;
border-radius: 5px;
}
```

```
@media (max-width: 700px) {
  .container {  flex-
direction: column;
  }
}
```

Script.js const studentForm =

document.getElementById("studentForm"); const fullName =

document.getElementById("fullName"); const email =

document.getElementById("email"); const program =

document.getElementById("program"); const semester =

document.getElementById("semester");

const studentCard = document.querySelector("#studentCard"); const

themeButton = document.querySelector("#themeButton");

const previewName = document.querySelector("#previewName");

const previewEmail = document.querySelector("#previewEmail"); const

previewProgram = document.querySelector("#previewProgram"); const

previewSemester = document.querySelector("#previewSemester");

const studentList = document.querySelector("#studentList");

fullName.addEventListener("input", function () {

previewName.textContent = fullName.value || "Your Name";

});

email.addEventListener("input", function () {

previewEmail.textContent = email.value || "Your Email";

});

program.addEventListener("input", function () {

previewProgram.textContent = program.value || "Your Program";

});

```
semester.addEventListener("input", function () {  
    previewSemester.textContent = semester.value || "Your Semester";  
});
```

```
studentForm.addEventListener("submit", function (event) {
```

```
    event.preventDefault();
```

```
    document.querySelector("#nameError").textContent = "";  
    document.querySelector("#emailError").textContent = "";  
    document.querySelector("#programError").textContent = "";  
    document.querySelector("#semesterError").textContent = "";
```

```
    let isValid = true;
```

```
    if (fullName.value.trim() === "") {  
        document.querySelector("#nameError").textContent = "Full Name is required.";  
        isValid = false;  
    }
```

```
    if (email.value.trim() === "") {  
        document.querySelector("#emailError").textContent = "Email is required.";  
        isValid = false;  
    } else if (!email.value.includes("@")) {  
        document.querySelector("#emailError").textContent = "Enter a valid email.";  
        isValid = false;  
    }
```

```
    if (program.value === "") {        document.querySelector("#programError").textContent  
= "Please select a program.";          
        isValid = false;
```

```

    }

    if (semester.value === "") {
document.querySelector("#semesterError").textContent = "Semester is required.";
        isValid = false;
    } else if (semester.value < 1 || semester.value > 8) {
        document.querySelector("#semesterError").textContent = "Semester must be between 1 and
8.";    isValid =
false;
    }

    if (isValid) {

        const listItem = document.createElement("li");

        listItem.textContent =
fullName.value + " | " +
email.value + " | " +    program.value
+ " | Semester " +    semester.value;

        studentList.appendChild(listItem);

        studentForm.reset();

        previewName.textContent = "Your Name";
        previewEmail.textContent = "Your Email";
        previewProgram.textContent = "Your Program";
        previewSemester.textContent = "Your Semester";
    }
});

```

```
themeButton.addEventListener("click", function () {  
  studentCard.classList.toggle("dark");  
});
```

OUTPUT :

Student Registration

Full Name

Email

Program

Semester

Register Student

Toggle Dark Mode

Student Card

Name: AMMANULLAH
Email: amman@gmail.com
Program: BS Computer Science
Semester: 4

Registered Students

Student Registration

Full Name

Email

Program

Semester

Register Student

Toggle Dark Mode

Student Card

Name: Your Name
Email: Your Email
Program: Your Program
Semester: Your Semester

Registered Students

AMMANULLAH | amman@gmail.com | BS Computer Science | Semester 4

Part B: Assignment
Task:

Interactive To-Do List

Build a To-Do

List application using only HTML, CSS and vanilla JavaScript.

Requirements

-

Add tasks through a form; empty or duplicate tasks must be rejected with a visible error message.

-

Each task has a Complete checkbox (strike-through style via classList) and a Delete button.

-

Show a live counter: total tasks and remaining tasks.

-

Add filter buttons: All, Active, Completed.

-

Use event delegation on the list container (one listener, not one per item).

-

Include a "Clear completed" button.

Bonus (optional)

-

Double-click a task to edit it inline.

-

Add a keyboard shortcut: press Enter to add, Escape to cancel editing.

CODE :

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Interactive To-Do List</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="todo-container">

    <h1>Interactive To-Do List</h1>

    <form id="todoForm">

      <input type="text" id="taskInput" placeholder="Enter a task">

      <button type="submit">Add Task</button>

    </form>

    <p id="errorMessage" class="error"></p>

    <div class="counter">

      <span>Total: <b id="totalTasks">0</b></span>

      <span>Remaining: <b id="remainingTasks">0</b></span>

    </div>

    <div class="filters">

      <button type="button" class="filter active" data-filter="all">All</button>

      <button type="button" class="filter" data-filter="active">Active</button>
```

```
        <button type="button" class="filter" data-filter="completed">Completed</button>
    </div>
```

```
<ul id="taskList"></ul>
```

```
<button id="clearCompleted" type="button">
    Clear Completed
</button>
```

```
</div>
```

```
<script src="script.js"></script>
</body>
</html>
```

Style.css

```
* {  box-sizing: border-
box;
}
```

```
body {  margin: 0;  font-
family: Arial, sans-serif;
background-color: #eeeeee;
}
```

```
.todo-container {
width: 90%;  max-
width: 700px;  margin:
50px auto;  padding:
30px;  background-
color: white;  border-
radius: 10px;  box-
```

```
shadow: 0 3px 12px
  rgba(0, 0, 0, 0.2);
}
```

```
h1 {  text-align:
center;
}
```

```
#todoForm {
  display: flex;
gap: 10px;
}
```

```
#taskInput {
  flex: 1;
  padding: 12px;
border: 1px solid #aaa;
border-radius: 5px;
}
```

```
#todoForm button {
padding: 12px 20px;
border: none;
background-color: #333;
color: white;  border-
radius: 5px;  cursor:
pointer;
}
```

```
.error {  color:
red;  min-height:
20px;
```

```
}
```

```
.counter {  display: flex;  
justify-content: space-between;  
margin: 20px 0;  
}
```

```
.filters {  display: flex;  
gap: 10px;  margin-  
bottom: 20px;  
}
```

```
.filter {  
flex: 1;  
padding: 10px;  
border: none;  border-  
radius: 5px;  cursor:  
pointer;  background-  
color: #ddd;  
}
```

```
.filter.active {  background-  
color: #333;  color: white;  
}
```

```
#taskList {  list-  
style: none;  
padding: 0;  
}
```

```
.task-item {  display: flex;  
align-items: center;  gap:
```

```
10px; padding: 12px;
margin-bottom: 8px;
background-color: #f1f1f1;
border-radius: 5px;
}
```

```
.task-text { flex:
1; cursor:
pointer;
}
```

```
.task-item.completed .task-text {
text-decoration: line-through; color:
gray;
}
```

```
.delete-button { border:
none; background-color:
#555; color: white;
padding: 7px 12px;
border-radius: 5px;
cursor: pointer;
}
```

```
.edit-input {
flex: 1;
padding: 7px;
}
```

```
#clearCompleted {  
width: 100%; padding:  
11px; border: none;  
border-radius: 5px;  
background-color: #333;  
color: white; cursor:  
pointer;  
}
```

```
@media (max-width: 600px) {  
#todoForm { flex-  
direction: column;  
}
```

```
.filters { flex-  
direction: column;  
}  
}
```

```
Script.js const todoForm =  
document.getElementById("todoForm"); const taskInput =  
document.getElementById("taskInput"); const taskList =  
document.getElementById("taskList"); const errorMessage  
= document.getElementById("errorMessage"); const  
totalTasks = document.getElementById("totalTasks"); const  
remainingTasks =  
document.getElementById("remainingTasks"); const  
clearCompleted =  
document.getElementById("clearCompleted");  
  
const filters = document.querySelector(".filters");
```

```
let currentFilter = "all";
```

```
todoForm.addEventListener("submit", function (event) {
```

```
    event.preventDefault();
```

```
    const taskText = taskInput.value.trim();
```

```
    if (taskText === "") {        errorMessage.textContent =  
"Task cannot be empty."  
        return;  
    }
```

```
    const tasks = document.querySelectorAll(".task-text");
```

```
    for (let task of tasks) {    if (task.textContent.toLowerCase() ===  
taskText.toLowerCase()) {        errorMessage.textContent =  
"Duplicate task is not allowed."  
        return;  
    }  
}
```

```
    errorMessage.textContent = "";
```

```
    const listItem = document.createElement("li");  listItem.classList.add("task-  
item");
```

```
    const checkbox = document.createElement("input");  
checkbox.type = "checkbox";  checkbox.classList.add("complete-  
checkbox");
```

```
    const taskSpan = document.createElement("span");  
taskSpan.classList.add("task-text");  taskSpan.textContent  
= taskText;
```

```
    const deleteButton = document.createElement("button");  
deleteButton.classList.add("delete-button");  deleteButton.textContent  
= "Delete";
```

```
    listItem.appendChild(checkbox);  
listItem.appendChild(taskSpan);  
listItem.appendChild(deleteButton);
```

```
taskList.appendChild(listItem);
```

```
taskInput.value = "";
```

```
    updateCounter();  
    applyFilter();  
});
```

```
taskList.addEventListener("click", function (event) {  
  
    if (event.target.classList.contains("complete-checkbox")) {  
        const listItem = event.target.parentElement;  
  
        listItem.classList.toggle("completed");  
  
        updateCounter();  
        applyFilter();  
    }  
}
```

```
if (event.target.classList.contains("delete-button")) {

    const listItem = event.target.parentElement;

    listItem.remove();

    updateCounter();
}
});

filters.addEventListener("click", function (event) {

    if (!event.target.classList.contains("filter")) {
return;
    }

    const buttons = document.querySelectorAll(".filter");

    buttons.forEach(function (button) {
        button.classList.remove("active");
    });
    event.target.classList.add("active");

    currentFilter = event.target.dataset.filter;

    applyFilter();
});

function applyFilter() {

    const tasks = document.querySelectorAll(".task-item");
```

```
tasks.forEach(function (task) {

    const isCompleted = task.classList.contains("completed");

    if (currentFilter === "all") {
task.style.display = "flex";
    }

    if (currentFilter === "active") {
if (isCompleted) {
task.style.display = "none";
    } else {
task.style.display = "flex";
    }
    }

    if (currentFilter === "completed") {
if (isCompleted) {
task.style.display = "flex";
    } else {
        task.style.display = "none";
    }
    }
});
}
```

```
function updateCounter() {

    const tasks = document.querySelectorAll(".task-item");
```

```
let remaining = 0;

tasks.forEach(function (task) {

    if (!task.classList.contains("completed")) {
remaining++;
    }

});

totalTasks.textContent = tasks.length;
remainingTasks.textContent = remaining;
}

clearCompleted.addEventListener("click", function () {

    const completedTasks = document.querySelectorAll(".task-item.completed");

    completedTasks.forEach(function (task) {
        task.remove();
    });

    updateCounter();
});

taskList.addEventListener("dblclick", function (event) {

    if (!event.target.classList.contains("task-text")) {
return;
    }

    const taskSpan = event.target;
    const oldText = taskSpan.textContent;
```

```
const editInput = document.createElement("input");

editInput.type = "text";
editInput.classList.add("edit-input"); editInput.value
= oldText;

taskSpan.replaceWith(editInput);

editInput.focus();

editInput.addEventListener("keydown", function (event) {

    if (event.key === "Enter") {

        const newText = editInput.value.trim();

        if (newText !== "") {
            taskSpan.textContent = newText;
            editInput.replaceWith(taskSpan);    }

        }

        if (event.key === "Escape") {
            editInput.replaceWith(taskSpan);

            }

        });

});
```

OUTPUT :

Interactive To-Do List

Enter a task

Add Task

Total: 1

Remaining: 1

All

Active

Completed

☐ TASK 1

Delete

Clear Completed

Interactive To-Do List

Add Task

Total: **2**

Remaining: **1**

AllActiveCompleted☐ TASK3DeleteClear Completed

Interactive To-Do List

Add Task

Total: **2**

Remaining: **1**

AllActiveCompleted☒ TASK-1DeleteClear Completed

Interactive To-Do List

Add Task

Total: 1

Remaining: 1

All

Active

Completed

Clear Completed

Github link :